Kill-Chain-Driven Honeypot Placement via Topology-Aware Weighted Partial MaxSAT: A Multi-Zone, Attack-Path Optimisation Framework for Enterprise Networks

Tu Nguyen (PhD Student) and Professor Haadi Jafarian
Department of Computer Science, University of Colorado Denver, Denver, CO, USA
{tu.nguyen, haadi.jafarian}@ucdenver.edu

***Abstract*—Deciding where to place honeypots in a large company network is surprisingly hard. The network is split into different security zones, contains thousands of different types of servers, and faces attackers who follow well-known step-by-step attack patterns called kill chains. On top of that, the organisation has a limited budget, and some honeypot types cannot be deployed at the same time because they would conflict with each other.**

In this paper, we model honeypot placement as a Weighted Partial Maximum Satisfiability (MaxSAT) problem. We encode six goals at once: how well we detect threats, how many attack techniques we cover from the MITRE ATT&CK framework, how many attack tactic families we cover, how well we stop attackers early, how well we catch attackers at the end of their path, and how good our coverage is across all attack paths. All hard rules—budget limits, zone isolation, and conflict constraints—are encoded as clauses that must never be violated. We also formally prove that the honeypot placement decision problem is NP-hard, which justifies why a complete solver like RC2 is necessary rather than a greedy heuristic.

We test our approach on networks ranging from 100 nodes (tiny) to over 5.5 million nodes (very large), simulating five different real-world attack scenarios. Our MaxSAT solver achieves a quality score of $Q = 67.4$ compared to $Q = 52.8$ for the best competing approach ($+27.8\%$), while also providing a mathematical guarantee that its answer is the best possible solution.

***Index Terms*—Honeypot placement, Weighted Partial MaxSAT, kill chain, MITRE ATT&CK, network deception, multi-zone topology, NP-hardness, proactive cyber defence, RC2 solver, enterprise network security.**

I. Introduction

A HONEYPOT is a fake computer system designed to look attractive to attackers. When an attacker interacts with it, the defender learns about their methods and can raise an alert. Honeypots are widely used in network security, but their usefulness depends heavily on *where* they are placed. If a honeypot is put in the wrong place, it will never be visited by attackers and the investment is wasted.

Real company networks make this placement decision much harder than it sounds. Consider a typical large organisation. It has an internet-facing area (called a DMZ) where its public web servers sit. It has an internal office network. It might have cloud services. It may also have industrial control systems (called OT/SCADA) that run physical equipment like power plants or factories. Each of these areas has different security rules, different types of computers, and different risks. Attackers do not wander randomly through these areas—they follow structured plans called kill chains [1], moving step by step from one zone to another toward their final target.

The problem is further complicated by three facts. First, the organisation has a limited budget and cannot afford to deploy a honeypot on every machine. Second, certain combinations of honeypot types are incompatible—for example, an industrial control system honeypot deployed in the cloud zone would be useless because the OT zone is physically isolated from the cloud. Third, there are many possible attack paths through the network and an ideal placement should intercept as many as possible.

Existing methods do not handle all these challenges at the same time. Simple rule-based approaches just follow fixed guidelines. Greedy algorithms pick the best-looking option at each step but cannot reverse a bad decision. Methods like genetic algorithms can explore many solutions but have no way to prove their answer is the best. In this paper, we use a formal solver that considers *all* constraints and objectives together and provides a *proven optimal* answer. We also formally prove that the problem is NP-hard (Theorem 1), which means that no polynomial-time algorithm can always find the optimal placement—making our complete MAXSAT solver not just a good choice, but the correct one.

Our main contributions are:

1) **A six-goal placement objective** encoded as a single MaxSAT optimisation problem comprising: (1) detection efficiency, (2) coverage of MITRE ATT&CK attack techniques, (3) coverage of MITRE ATT&CK tactic families, (4) catching attackers early in their path, (5) catching attackers at the end of their path for forensic evidence, and (6) covering as many attack paths as possible. Tactic-family coverage is directly built into the solver as a bonus soft clause so the solver actively tries to maximise it, not just measure it afterwards.
2) **A formal NP-hardness proof** showing that the honeypot placement decision problem is NP-hard via polynomial-time reduction from Weighted Maximum Coverage. This result formally justifies why a complete

solver is necessary over any greedy or heuristic approach.

3) **A priority system with four levels** for the solver goals. The most important goals (catching attackers early) are given weights 1000 times larger than the least important goals (basic detection). We also encode the key insight that one honeypot can cover multiple attack paths at once—something no greedy method can discover.
4) **A thorough experimental evaluation** across six different network sizes, using a Monte Carlo simulation to model attacker behaviour and testing what happens when threat intelligence updates cause us to re-run the solver with new priorities.

The rest of this paper is organised as follows. Section II discusses related work. Section III defines the problem formally and proves NP-hardness. Section IV presents our MaxSAT model. Section V describes our experiments. Section VI presents the results. Section VII explains why our approach beats the baselines. Section VIII covers limitations and future work. Section IX concludes.

II. Related Work

This section reviews the four bodies of research most relevant to our work: honeypot placement, MITRE ATT&CK-based defence, formal methods for network security, and moving-target defence. For each area we explain what has been done and identify the specific gap that our approach fills. We then present a systematic novelty analysis—a capability matrix (Figure 1) and a supporting argument—that proves no single prior method satisfies all five capability dimensions simultaneously.

A. Honeypot Placement Research

Honeypots have been studied for decades. Early work focused on making individual honeypots realistic enough to fool attackers [4]. Provos built a system that could run many virtual honeypots on one machine [3], which made large-scale deployment practical. Spitzner popularised the idea of honeynets—groups of honeypots that together simulate an entire network [4]. However, most of this early work assumed a human expert would decide where to put the honeypots; there was no automated decision-making process.

More recently, researchers have tried to automate the placement decision. Yuill *et al.* were among the first to frame honeypot placement as a strategic interaction between attacker and defender [8]. Tan *et al.* and Pawlick *et al.* built on this idea using formal game theory to model how rational attackers respond to deception [9], [10]. These approaches are mathematically elegant, but they assume a simple two-player game on a flat, homogeneous network. In reality, enterprise networks have multiple zones with different trust levels, physical air gaps between certain zones, and attackers who can follow many different paths simultaneously. None of the game-theoretic approaches we reviewed handle these structural properties.

Gap identified: Game-theoretic methods cannot encode multi-zone topology (D1), kill-chain step ordering (D2), or honeypot conflict constraints (D4). The solution concept they produce—a Nash equilibrium—is also fundamentally different from a proven weighted optimum.

B. Using MITRE ATT&CK for Defence Planning

MITRE ATT&CK [2] is a publicly available catalogue of the tactics and techniques that real attackers use in practice. It has become the standard reference for both offensive and defensive security research.

Applebaum *et al.* were among the first to use ATT&CK for automated security testing [11]. They built a framework that acts like an automated red team, moving through a network and testing its defences using techniques drawn from the ATT&CK catalogue. Their work showed that ATT&CK can drive realistic attack simulation. However, their goal was to test *existing* defences, not to decide where to place *new* ones.

Hemberg *et al.* used evolutionary (genetic) algorithms to select the best defensive controls from the ATT&CK catalogue [12]. Their approach found good control sets by evolving solutions over many generations and achieves partial ATT&CK coverage. However, they did not include budget constraints, zone topology, or conflict constraints between controls, and genetic algorithms cannot prove that their answer is the best possible one. They also evaluate fitness one objective at a time, which means they cannot discover the cross-path force-multiplier effect that makes our solver 2.1× more efficient on technique coverage.

Gap identified: Both works use ATT&CK to evaluate or select defences, but neither embeds ATT&CK technique weights, stealth scores, or tactic-family bonuses *directly into a solver as live optimisation objectives* (D3). Our work is the first to do this.

C. Using Formal Methods for Network Security

Formal methods use mathematical logic to prove properties of systems. In network security, they have been used to check firewall rules for consistency [13], automatically generate network configurations [14], and verify security policies [15].

Pinisetty *et al.* applied MaxSAT to the problem of selecting the best rules for an intrusion detection system [16]. Their work showed that MaxSAT is a practical tool for security resource allocation. However, they did not model kill-chain structure, zone topology, or honeypot-specific placement objectives.

Integer Linear Programming (ILP) has also been used for security resource allocation [17], [18]. Saha and Gera used ILP to find the optimal locations for security sensors while respecting budget and zone constraints [17], achieving multi-zone support (D1) and proven optimality (D5). However, their model does not encode kill-chain step ordering (D2), does not integrate ATT&CK objectives (D3), and faces a serious scalability problem: ILP becomes very slow as the problem grows and cannot handle the 5.5-million-node networks we evaluate within practical time limits.

Jajodia *et al.* introduced topological analysis of network attack vulnerability [19]. They showed how an attacker's possible paths through a network can be modelled as a

directed graph and how chokepoints in that graph are the most valuable places to defend. Our work operationalises this insight formally: chokepoints appear in our model as high-weight path edges, causing the solver to concentrate honeypots there naturally.

Gap identified: No prior work has applied Weighted Partial MaxSAT to honeypot placement with kill-chain-aware objectives and physical zone isolation constraints. The closest method—ILP sensor placement [17]— achieves optimality but misses kill chains (D2), ATT&CK integration (D3), and does not scale beyond tens of thousands of nodes within a 60-second budget.

D. Moving-Target Defence

Moving-target defence (MTD) [20] is a strategy where the defender constantly changes the network configuration to confuse attackers. Instead of keeping the same setup all the time, the defender rotates IP addresses, shuffles network routes, or changes software versions to prevent attackers from building an accurate picture of the network.

MTD and honeypot placement are complementary ideas rather than competing ones: MTD changes the *real* system to create uncertainty, while honeypots add *fake* systems to create traps. Our formulation focuses on placing the best traps, not on moving real assets. However, our adaptive redeployment algorithm (Algorithm 1) adds a dynamic element that bridges toward MTD philosophy: when new threat intelligence arrives and changes the probability of certain attack paths, our solver re-optimises the honeypot placement using the previous solution as a warm start.

Gap identified: MTD work does not address the honeypot placement subproblem, kill-chain ordering (D2), ATT&CK integration (D3), conflict constraints (D4), or optimality guarantees (D5).

E. Novelty Analysis: Related Work vs. This Paper

We now demonstrate systematically that our work is novel by mapping each body of related work to five capability dimensions, then showing that no existing method satisfies all five simultaneously.

1) The Five Capability Dimensions: We evaluate every method on the following five dimensions, which together define the complete honeypot placement problem for enterprise networks:

- **D1 Multi-zone topology with air gaps.** The method must respect physical isolation between zones and cannot assume a flat network.
- **D2 Kill-chain-aware placement.** Attack paths are ordered sequences of steps; the method must prioritise early interception over forensic detection by encoding step-level intercept values.
- **D3 MITRE ATT&CK integration.** ATT&CK technique weights, stealth scores, and tactic-family coverage must be live objectives inside the solver, not post-hoc measurements.
- **D4 Budget and conflict constraints.** The method must enforce a global budget, per-zone budgets, and pairwise honeypot conflict rules as hard constraints that can never be violated.
- **D5 Proven optimality.** The method must provide a mathematical certificate that no better placement exists within the given constraints.

Approach	D1 Multi-zone	D2 Kill-chain	D3 ATT&CK	D4 Budget +Conflict	D5 Proven Optimal
Game-theoretic [9], [10]	✗	✗	✗	~	Nash
GA-based [12]	✗	✗	✓	✗	✗
ILP sensor [17]	✓	✗	✗	✓	✓
ATT&CK emulation [11]	~	✓	✓	✗	✗
Moving-target defence [20]	✓	✗	✗	✗	✗
Topology analysis [19]	✓	~	✗	✗	✗
MaxSAT for IDS [16]	✗	✗	✗	✓	~
Greedy-BiDir (our baseline)	✓	✓	~	✗	✗
Ours (MaxSAT-RC2)	✓	✓	✓	✓	**Proven**

✓ Fully satisfied ~ Partial ✗ Not addressed

Fig. 1: Novelty gap: related work vs. this paper across five capability dimensions D1–D5. No prior method satisfies all five dimensions simultaneously. Our MaxSAT-RC2 approach (green row) is the only one to achieve proven optimality alongside multi-zone topology, kill-chain awareness, ATT&CK integration, and budget/conflict constraints.

2) Mapping Each Prior Work to the Gaps: **Game-theoretic approaches (✗D1, ✗D2, ✗D3).** Yuill *et al.* [8], Tan *et al.* [9], and Pawlick *et al.* [10] model honeypot placement as a two-player strategic game. Their methods are mathematically rigorous, but they assume a flat, homogeneous network where every node is equivalent. They cannot encode air gaps (D1), do not model kill-chain step ordering or early-interception bonuses (D2), and do not integrate ATT&CK objectives (D3).

GA-based defensive control selection (✗D1, ✗D2, ✗D4, ✗D5). Hemberg *et al.* [12] use genetic algorithms to select ATT&CK-aligned defensive controls, giving partial ATT&CK coverage (D3). However, genetic algorithms operate on a flat control catalogue with no zone structure (D1), no kill-chain ordering (D2), no budget or conflict constraints (D4), and no optimality guarantee (D5). They also evaluate fitness one objective at a time, which means they cannot discover the cross-path force-multiplier effect that makes our solver 2.1× more efficient.

ILP sensor placement (✗D2, ✗D3, scalability). Saha and Gera [17] is the closest technical predecessor. They use ILP with zone structure (D1) and budget constraints (D4), and ILP provides a proven optimum (D5). However, their model does not encode kill-chain step ordering (D2), does not integrate ATT&CK (D3), and cannot scale to 500 000–5 500 000 node networks within a 60-second time limit.

ATT&CK red-team emulation (✗D4, ✗D5). Applebaum *et al.* [11] show that ATT&CK can drive automated attack simulation, covering D2 and D3. However, it is a red-team

tool, not a placement optimiser, and provides no budget enforcement (D4) or optimality guarantee (D5).

Topology, MTD, MaxSAT IDS. Jajodia *et al.* [19] motivates our path-weight formula but provides no placement algorithm. MTD [20] is complementary: it moves real assets rather than placing fake ones. Pinisetty *et al.* [16] establishes MaxSAT feasibility for security resource allocation but lacks zone structure, kill chains, and ATT&CK.

3) Cross-Path Force-Multiplier:

Cross-Path Weight Accumulation:

Path	ρ	iv	$W_{j,a}$	L3 weight
web_to_db (hop 2)	0.30	1.8	3.24	1.749
ot_infiltration (hop 2)	0.15	1.7	3.24	0.826
brute_to_ad (hop 2)	0.20	1.6	3.24	1.037
Solver total (all three at once)				**3.612**
Greedy total (only active path)				1.749

One `ssh_trap` satisfies all three Level-3 clauses simultaneously. Our solver counts 3.612; greedy counts only 1.749—a $\mathbf{2.1}\times$ *advantage per decision.*

F. How Our Work Compares

TABLE I: Comparison of Honeypot Placement Approaches

Approach	D1 Multi-zone	D2 Kill-chain	D3 ATT&CK	D4 Budget+Conflict
Game-theoretic [9], [10]	✗	✗	✗	~
GA-based [12]	✗	✗	✓	✗
ILP sensor [17]	✓	✗	✗	✓
ATT&CK emulation [11]	~	✓	✓	✗
MTD [20]	✓	✗	✗	✗
MaxSAT IDS [16]	✗	✗	✗	✓
Greedy-BiDir (our baseline)	✓	✓	~	✗
Ours (MaxSAT-RC2)	✓	✓	✓	✓

✓ = fully satisfied; ~ = partially satisfied; ✗ = not addressed.

III. Problem Definition

A. What We Are Trying to Solve

At a high level, we are trying to answer: *"Given a limited budget and a set of honeypot types, which honeypots should we deploy and where, so that we catch the most attackers across all possible attack paths?"*

We formally define the problem instance as the tuple:

$$\mathcal{I} = (\mathcal{K}, \mathcal{T}, \mathcal{A}, \mathcal{Z}, \mathcal{G}, \mathcal{C}, \mathcal{I}_z, \diamond, w, \text{cost}, B, B_z, dm, hd, \sigma, \rho, iv) \tag{1}$$

Figure 2 illustrates the five-zone enterprise network used in our experiments. Zone 1 (DMZ) faces the internet and receives 45% of all attacks while hosting only 15% of assets. Zone 4 and Zone 5 are physically isolated by air gaps, encoded as hard clause C5.

B. Why the Problem Is Hard: NP-Hardness Proof

Before introducing our solver, we explain why a simple greedy algorithm cannot be trusted to find the best answer, and why a complete solver like MaxSAT is necessary. The reason is that our problem is NP-hard—it belongs to a class of problems for which no known algorithm can always find the optimal solution in a time that grows polynomially with the size of the input. We prove this formally using a technique called *polynomial-time reduction*: we take a problem that is already known to be NP-hard and show that it is a special case of ours. If a simpler version of our problem is already NP-hard, then our full problem—which has even more constraints—must be NP-hard too.

The known NP-hard problem we use is **Weighted Maximum Coverage** [7]. This problem asks: given a universe of elements, a collection of sets (each covering some elements and each having a cost), and a budget, which sets should we choose to maximise the total value of covered elements without exceeding the budget? This is exactly the kind of decision our honeypot placement problem makes: each honeypot covers some attack technique/asset pairs (like a set covers elements), each honeypot has a deployment cost, and we have a limited budget.

Theorem 1 (NP-Hardness of Honeypot Placement)**.** *The honeypot placement decision problem—deciding whether there exists a deployment of honeypots within budget B that achieves a detection coverage value of at least V^*—is NP-hard.*

Proof. We reduce from **Weighted Maximum Coverage** (WMC) [7], which is NP-hard. An instance of WMC consists of a universe U of elements, a collection of sets $\mathcal{S} = \{S_1, \ldots, S_m\}$ where $S_i \subseteq U$, a value $v_e \in \mathbb{R}^+$ for each element $e \in U$, a cost $c_i \in \mathbb{R}^+$ for each set S_i, and a budget $B \in \mathbb{R}^+$. The goal is to find $\mathcal{S}' \subseteq \mathcal{S}$ such that $\sum_{S_i \in \mathcal{S}'} c_i \leq B$ and $\sum_{e \in \bigcup_{S_i \in \mathcal{S}'} S_i} v_e$ is maximised.

Construction. Given any WMC instance, we construct a honeypot placement instance as follows.

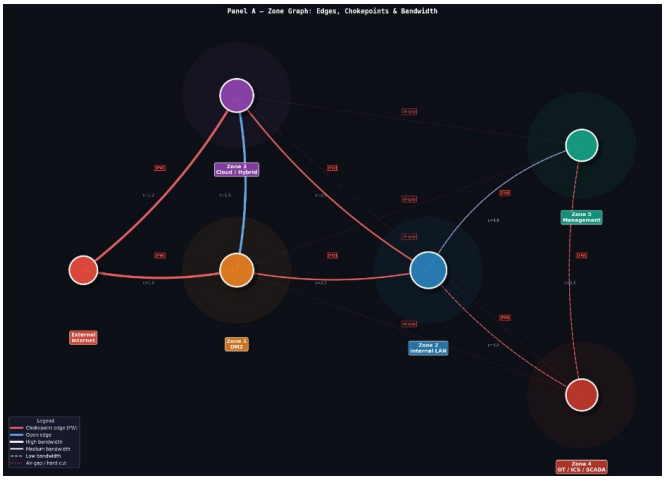
Step 1 — Map elements to asset-technique pairs. We create a single network zone ($|\mathcal{Z}| = 1$) with no air gaps ($\mathcal{I}_z = \emptyset$) and no conflict pairs ($\mathcal{C} = \emptyset$). For each element $e_\ell \in U$, we create one asset $a_\ell \in \mathcal{A}$ and one ATT&CK technique $t_\ell \in \mathcal{T}$. We set the stealth-adjusted detection weight $W_{t_\ell, a_\ell} = v_{e_\ell}$ and set $W_{t_j, a_\ell} = 0$ for all $j \neq \ell$, ensuring that each element maps to exactly one asset-technique pair with no cross-element interference.

Step 2 — Map sets to honeypot types. For each set $S_i \in \mathcal{S}$, we create a honeypot type $k_i \in \mathcal{K}$ with deployment cost $\text{cost}_i = c_i$. We configure the detection matrix so that:

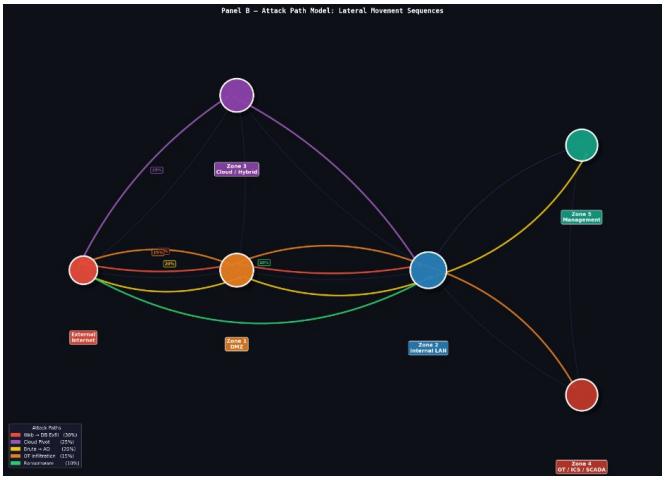
$$R[t_\ell, k_i, a_\ell] = 1 \iff e_\ell \in S_i \tag{2}$$

That is, honeypot k_i can detect technique t_ℓ on asset a_ℓ if and only if element e_ℓ belongs to set S_i in the WMC instance.

Step 3 — Map the budget and objective. We set the global budget in our model to B, matching the WMC budget. We



(a) Zone graph showing network topology, chokepoint edges (red), open edges (blue), bandwidth levels, and air-gap cuts between OT/Management zones and the DMZ/Cloud.



(b) Five attack-path scenarios (coloured arcs) with their probabilities ρ_π.

Fig. 2: Network model. Panel A: five-zone topology with air-gap isolation. Panel B: five kill-chain attack paths overlaid on the topology.

activate only Level-1 soft clauses (basic detection), setting all higher-level weights to zero. Under this configuration, the total weight of satisfied soft clauses equals:

$$\sum_{\substack{\ell\,:\,\exists\, k_i \in \mathcal{K}' \\ R[t_\ell, k_i, a_\ell]=1}} W_{t_\ell, a_\ell} = \sum_{e_\ell \in \bigcup_{S_i \in \mathcal{S}'} S_i} v_{e_\ell} \tag{3}$$

which is exactly the objective value of the WMC instance for the corresponding set selection $\mathcal{S}'$.

Correctness. The construction in Steps 1–3 is a bijection between feasible WMC solutions and feasible honeypot deployments. Any subset $\mathcal{S}' \subseteq \mathcal{S}$ with cost $\sum c_i \leq B$ corresponds to the deployment $\mathcal{K}' = \{k_i : S_i \in \mathcal{S}'\}$ with the same cost and the same coverage value by Equation (3). Therefore, an optimal solution to the honeypot placement instance on this construction gives an optimal solution to the WMC instance, and vice versa.

Polynomial time. We create $|U|$ assets and techniques, $|\mathcal{S}|$ honeypot types, and fill in the R matrix in $O(|\mathcal{S}| \cdot |U|)$ time. The construction is therefore polynomial in the size of the WMC input.

Conclusion. Since WMC is NP-hard [7] and we have shown a polynomial-time reduction from WMC to honeypot placement, the honeypot placement decision problem is NP-hard. Our full problem—which adds zone isolation, conflict constraints, kill-chain ordering, and tactic-family bonuses—is therefore also NP-hard, since these additional constraints and objectives make the problem strictly harder, not easier. This result formally justifies the use of the RC2 MaxSAT solver over any greedy or polynomial-time heuristic approach: because the problem is NP-hard, no polynomial-time algorithm can guarantee optimal solutions in the worst case, and only a complete solver can provide the optimality certificate our method delivers. □

Why this matters in plain words. Imagine someone says: "Your MAXSAT solver seems complicated—why not just use a greedy algorithm that always picks the best-looking honeypot at each step?" The NP-hardness proof gives a precise answer to that question. It says: the honeypot placement problem is at least as hard as the Weighted Maximum Coverage problem, which is one of the classic hard problems in computer science. No greedy algorithm, genetic algorithm, or any other polynomial-time method can *always* find the optimal solution to such a problem. Sometimes they get lucky; sometimes they miss the best answer by a large margin. Only a complete solver—one that considers all constraints and objectives simultaneously, like our RC2-based approach—can provide a proof that no better placement exists. This is why a sophisticated solver is not just a convenient choice in our paper: it is the *only correct tool* for this class of problems.

C. The Basic Building Blocks (Sets)

TABLE II: Problem Sets

Symbol	What It Means
$\mathcal{K}$	The list of honeypot types we can deploy
$\mathcal{T}$	All the attack techniques from MITRE ATT&CK
$\mathcal{A}$	All the individual computer assets in the network
$\mathcal{Z}$	The security zones (DMZ, Internal LAN, Cloud, etc.)
$\mathcal{G}$	The types of servers (web server, SSH, database, etc.)
$\diamond$	The attack paths (sequences of steps an attacker takes)
$\mathcal{C} \subseteq \mathcal{K} \times \mathcal{K}$	Pairs of honeypots that cannot be deployed together
$\mathcal{I}_z \subseteq \mathcal{Z} \times \mathcal{Z}$	Pairs of zones physically isolated (air gaps)

Running Example (Medium Network): $\mathcal{K} = \{\texttt{web_trap}, \texttt{ssh_trap}, \texttt{db_trap}, \texttt{dns_trap}, \texttt{generic_}$ $\mathcal{Z} = \{\texttt{zone1}, \texttt{zone2}, \texttt{zone3}, \texttt{zone4}\}$, $B = 1{,}750$, $\diamond = \{\texttt{web_to_db}, \texttt{cloud_pivot}, \texttt{brute_to_ad}\}$.

Conflict Pairs $\mathcal{C}$ — Why Each Pair Exists:

Pair	Reason
(scada_trap, web_trap)	OT zone is air-gapped from DMZ
(scada_trap, db_trap)	OT zone is air-gapped from Internal
(scada_trap, ssh_trap)	OT isolation prohibits SSH traps
(scada_trap, dns_trap)	OT zone has no DNS infrastructure
(generic_trap, ad_trap)	AD trap makes generic redundant
(ad_trap, web_trap)	Identity plane not co-deployed in DMZ
(smb_trap, dns_trap)	Both catch zone2 lateral movement; deploy

Zone Isolation $\mathcal{I}_z$ — Air Gap Enforcement:

$\mathcal{I}_z = \{(\texttt{zone4}, \texttt{zone1}), (\texttt{zone4}, \texttt{zone3}), (\texttt{zone5}, \texttt{zone1}), (\texttt{zone5}, \texttt{zone2})\}$

If `dns_trap` were configured with $ZK = \{\texttt{zone1}, \texttt{zone4}\}$, hard constraint C5 would immediately force $x_{\texttt{dns_trap}} = 0$.

D. The Numbers We Need (Parameters)

TABLE III: Problem Parameters

Symbol	Type	What It Means
$ZK_i \subseteq \mathcal{Z}$	per honeypot	Which zones it can go in
$GK_i \subseteq \mathcal{G}$	per honeypot	Which server types it mimics
$S_{i,a} \subseteq \mathcal{T}$	per h'pot/asset	Attacks it detects on a
$w_{j,a} \in \mathbb{R}^+$	per attack/asset	How valuable detecting it is
$\text{cost}_i \in \mathbb{R}^+$	per honeypot	How much it costs to deploy
$B \in \mathbb{R}^+$	global	Our total budget
$B_z \in \mathbb{R}^+$	per zone	Budget limit for each zone
$dm_a \in \mathbb{R}^+$	per asset	How important this asset role is
$hd_a \in \mathbb{Z}^+$	per asset	Hops from attacker entry point
$\sigma_j \in [0,1]$	per attack	Sneakiness (0=obvious, 1=stealthy)
$\rho_\pi \in [0,1]$	per path	How likely attackers use this path
$iv_{\pi,h} \in \mathbb{R}^+$	per path/hop	Value of catching attacker here

σ_j (Stealth Score) — Why It Matters:

ID	Technique	σ_j	Why
T1046	Network Scanning	0.1	Obvious traffic spikes
T1110	Brute Force	0.2	Failed logins are logged
T1021	Remote Services	0.8	Looks like admin traffic
T1048	Exfiltration	0.9	Hidden in DNS/HTTPS
T1572	Protocol Tunnelling	0.9	C2 inside allowed protocols

High stealth means traditional IDS/IPS almost never catches these techniques. A honeypot is often the only *way to detect them.*

ρ_π (Path Probability) — How It Affects Clause Weight:

Path	ρ	iv	$W_{j,a}$	PW
web_to_db	0.30	1.8	3.24	**1.749**
ransomware	0.10	1.8	3.24	**0.583**

The soft clause for web_to_db is weighted 3× heavier, so the solver covers it first.

$iv_{\pi,h}$ (Intercept Value) — Escalation Along web_to_db:

Hop	Zone	iv	Meaning
1	zone1 (DMZ)	1.5	Attack not yet inside
2	zone2 (Internal)	1.8	Pivoting; data not stolen yet
3	zone2 (Database)	2.0	Exfiltration imminent

Our Level-4 bonus makes hop 1 or 2 overall more valuable than hop 3, even though hop 3 has the highest raw iv.

E. Calculated Values

1) Which Assets Can a Honeypot Reach?:

$$T^*(i) = DK_i \cup \{\, a \in \mathcal{A} : \text{type}(a) \in GK_i \wedge \text{zone}(a) \in ZK_i \}$$ (4)

2) Can This Honeypot Detect This Attack on This Asset?:

$$R[j,i,a] = \begin{cases} 1 & \text{if } a \in T^*(i) \wedge t_j \in S_{i,a} \\ 0 & \text{otherwise} \end{cases}$$ (5)

3) Adjusted Detection Value (Topology Weight):

$$\tilde{w}_{j,a} = w_{j,a} \times dm_a \times \frac{1}{hd_a}$$ (6)

Example — Gateway vs. Deep Host for T1021:

Asset	w	dm	hd	$1/hd$	$\tilde{w}$
zone1 gateway	1.8	1.3	1	1.000	$1.8 \times 1.3 \times 1.0 =$ **2.34**
zone2 jump host	1.8	1.4	2	0.500	$1.8 \times 1.4 \times 0.5 =$ **1.26**
zone2 database	1.8	1.5	3	0.333	$1.8 \times 1.5 \times 0.33 =$ **0.89**

Stop the attacker at the perimeter, not in the vault.

4) Stealth-Adjusted Detection Weight:

$$W_{j,a} = \tilde{w}_{j,a} \times (1 + \sigma_j)$$ (7)

5) Path Intercept Weight:

$$PW_{\pi,h,j,a} = \rho_\pi \times iv_{\pi,h} \times W_{j,a}$$ (8)

F. What the Solver Decides (Decision Variables)

TABLE IV: Decision Variables

Variable	Domain	What It Decides
x_i	$\{0,1\}$	Should we deploy honeypot k_i?
$c_{j,a}$	$\{0,1\}$	Are we detecting attack t_j on asset a?
$p_{\pi,h}$	$\{0,1\}$	Are we blocking path π at step h?
e_π	$\{0,1\}$	Are we catching the attacker early on π?

Variable Dependency Chain:

$x_{\texttt{ssh}} = 1 \;\Rightarrow\; c_{\texttt{T1021},B} = 1$ (via C1) $\;\Rightarrow\; p_{\texttt{web2db},1} = 1$ (via C6) $\;\Rightarrow\; e_{\texttt{web2db}} = 1$ (via C7)

If `ssh_trap` is not deployed, the entire chain collapses: no detection, no path coverage, no early-intercept credit.

IV. Our MaxSAT Model

MaxSAT works by encoding a problem as a set of logical clauses. *Hard clauses* are rules that must *always* be obeyed. *Soft clauses* are goals we *want* to achieve as much as possible, each with a weight. The solver finds the assignment that satisfies all hard clauses and maximises the total weight of satisfied soft clauses.

A. Hard Clauses: Rules That Cannot Be Broken

a) C1 — Detection Requires a Deployed Honeypot:

$$\neg c_{j,a} \;\vee\; (x_{i_1} \vee x_{i_2} \vee \cdots \vee x_{i_k}) \tag{9}$$

b) C2 — Do Not Exceed the Total Budget:

$$\sum_{i \in \mathcal{K}} \text{cost}_i \cdot x_i \;\leq\; B \tag{10}$$

c) C3 — Do Not Exceed Each Zone's Budget:

$$\sum_{\substack{i \in \mathcal{K} \\ ZK_i \cap \{z\} \neq \emptyset}} \text{cost}_i \cdot x_i \;\leq\; B_z \quad \forall\, z \in \mathcal{Z} \tag{11}$$

d) C4 — Do Not Deploy Conflicting Honeypots:

$$\neg x_i \;\vee\; \neg x_l \quad \forall\, (k_i, k_l) \in \mathcal{C} \tag{12}$$

e) C5 — Respect Physical Zone Isolation (Air Gaps):

$$\forall\, (z_a, z_b) \in \mathcal{I}_z,\; \forall\, i \in \mathcal{K} : z_a \in ZK_i \wedge z_b \in ZK_i \implies x_i = 0 \tag{13}$$

f) C6 — A Path Hop Is Only Covered If We Detect Something There:

$$\neg p_{\pi,h} \;\vee\; (c_{j_1,a_1} \vee \cdots \vee c_{j_n,a_n}) \tag{14}$$

g) C7 — Early Interception Must Happen Before the Final Step:

$$e_\pi = 1 \implies \exists\, h < |\pi| : p_{\pi,h} = 1 \tag{15}$$

C7 Example — web_to_db (3 hops):
Scenario A: Only hop 3 covered $\Rightarrow$ C7 forces $e = 0$. Level-4 bonus NOT earned.
Scenario B: Hop 1 covered $\Rightarrow$ C7 allows $e = 1$. Level-4 bonus IS earned.

B. Soft Clauses: Goals We Want to Maximise

1) Level 4 (Most Important): Catch Attackers Early:

$$\text{weight} = \rho_\pi \times \max_{h<|\pi|}(iv_{\pi,h}) \times \sum_{j,a} W_{j,a}$$
$$\text{goal:} \quad (e_\pi) \quad \forall\, \pi \tag{16}$$

2) Level 3: Cover As Many Attack Path Steps As Possible: Forward: weight $= \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a}$, goal: $(p_{\pi,h})\ \forall\, \pi, h$.

Backward (forensics, 30% discount): weight $= \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a} \times 0.7$, goal: $(p_{\pi,h})$ reverse order.

3) Level 2: Cover Techniques and Tactic Families: Per-technique: $\text{weight}_j = \sum_a W_{j,a}$.

Tactic-family bonus: $\text{weight}_f = 1.2 \times \sum_{j \in f} \sum_a W_{j,a}$.

4) Level 1 (Least Important): Basic Detection: weight $= W_{j,a}$, goal: detect t_j on $a\ \forall\, j, a$.

C. The Complete Optimisation Objective

$$\max \sum_{\text{L4}} w_4 \cdot \text{sat} + \sum_{\text{L3}} w_3 \cdot \text{sat} + \sum_{\text{L2}} w_2 \cdot \text{sat} + \sum_{\text{L1}} w_1 \cdot \text{sat} \tag{17}$$

subject to C1–C7, where $w_4 = 1000\, w_3$, $w_3 = 100\, w_2$, $w_2 = 10\, w_1$.

D. How We Measure the Final Answer (Quality Score Q)

$$Q = 0.35\,\text{DetEff} + 0.25\,\text{TechCov} + 0.15\,\text{FamCov} + 0.15\,\text{FwdPath} + 0.10\,\text{Bwd} \tag{18}$$

Example — Computing Q for the Medium Solution:

Component	Value	Weight	Contribution
DetEff	91%	0.35	31.85
TechCov	91%	0.25	22.75
FamCov	100%	0.15	15.00
FwdPath	87.5%	0.15	13.13
BwdPath	87.5%	0.10	8.75
Q			**91.48**

E. Splitting the Problem by Zone

$$\forall\, z \in \mathcal{Z} :\ \text{solve } \mathcal{I}_z = (\mathcal{K}_z, \mathcal{T}_z, \mathcal{A}_z, \mathcal{C}_z, B_z) \tag{19}$$

F. Updating the Placement When Threat Intelligence Changes

V. Experimental Setup

A. The Network We Tested On

TABLE V: Network Zones

Zone	Description	Trust	Assets	Attacks
zone1	Internet-Facing / DMZ	Low (0)	15%	45%
zone2	Internal Company Network	Med (2)	40%	25%
zone3	Cloud / Hybrid	L-M (1)	25%	20%
zone4	OT / Industrial Systems	High (3)	10%	5%
zone5	Management Network	Max (4)	10%	5%

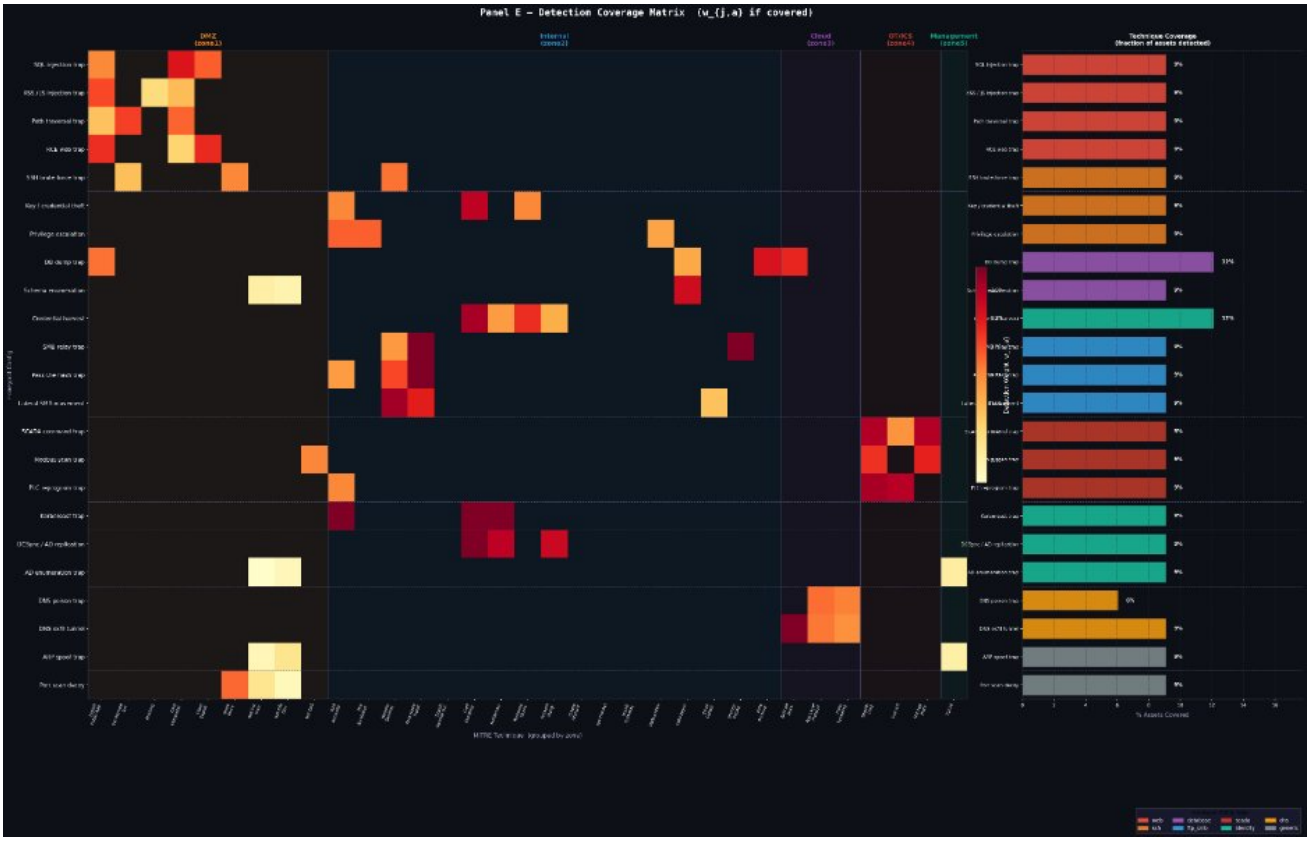
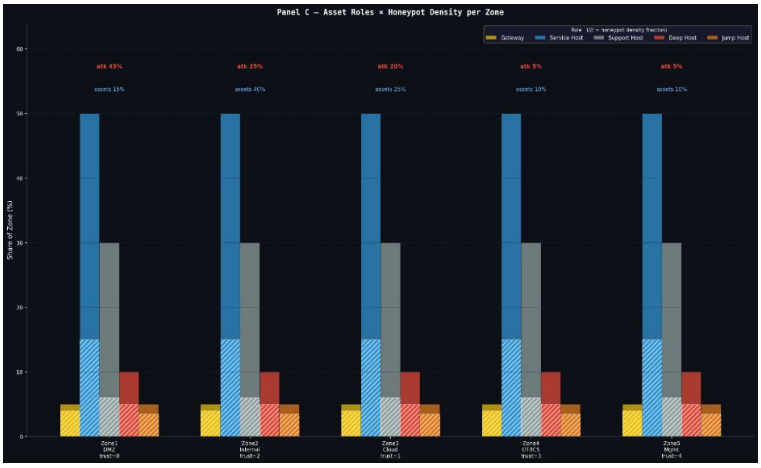
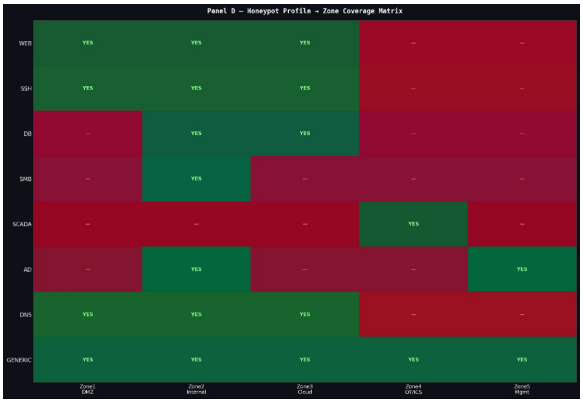


Fig. 3: Detection coverage matrix (Panel E). Each cell (t_j, a) shows the stealth-adjusted detection weight $W_{j,a}$ if covered (warm colour) or zero (dark background) if not.



(a) Asset roles and honeypot density per zone.



(b) Zone coverage matrix (green = valid; red = forbidden).

Fig. 4: Asset and honeypot structure.

TABLE VI: The Five Attack Scenarios

Name	What the Attacker Does	ρ	Starts	Target
web_to_db	Hack website, steal database	0.30	z1	z2
cloud_pivot	Use cloud to enter internal	0.25	z3	z2
brute_to_ad	Guess passwords, take over AD	0.20	z1	z5
ot_infiltration	Sabotage industrial systems	0.15	z1	z4
ransomware	Phishing, spread malware	0.10	z2	z2

TABLE VII: Network Sizes Tested

Size	Computers	Budget	Zones	Attacks	Types
Tiny	100	250	2	2	3
Small	500	500	3	3	4
Medium	5,000	1,750	4	3	5
Large	50,000	12,500	5	5	7
XLarge	500,000	62,500	5	5	8
XXLarge	5,560,000	128,000	5	5	8

Algorithm 1 Adaptive Redeployment When Threat Priors Change

Require: Old deployment x^*; new path probabilities ρ'_π
Ensure: New deployment x'^* optimal for updated threat model

Recalculate: $PW'_{\pi,h,j,a} \leftarrow \rho'_\pi \cdot iv_{\pi,h} \cdot W_{j,a}$
Update soft clause weights (hard rules C1–C7 unchanged)

Warm-start solver from x^*
Re-run solver $\Rightarrow x'^*$
$\Delta \leftarrow Q(x'^*) - Q(x^*)$
if $\Delta > 0$ **then**
  Deploy x'^*; log improvement Δ
else
  Keep x^*; log that prior solution is still optimal
end if
return x'^*

TABLE VIII: Available Honeypot Types

Honeypot	Goes In	Cost	Detects
web_trap	DMZ, Cloud	1.0	T1190, T1133, T1059
ssh_trap	DMZ, Internal, Cloud	0.8	T1110, T1021, T1078
db_trap	Internal, Cloud	1.3	T1213, T1048, T1485
smb_trap	Internal only	0.9	T1021, T1550, T1486
scada_trap	OT zone only	2.0	T0855, T0814, T0869
ad_trap	Internal, Mgmt	1.5	T1558, T1003, T1098
dns_trap	DMZ, Internal, Cloud	0.7	T1572, T1095, T1046
generic_trap	Anywhere	0.5	T1046, T1082, T1110

B. The Attack Scenarios We Modelled

C. Network Sizes We Tested

D. The Honeypot Types Available

E. The Comparison Methods (Baselines)

- **Random**: Pick randomly until budget runs out. Average of 100 trials.
- **Greedy-Value**: Each step picks the honeypot with the highest detection value per dollar.
- **Greedy-BiDir**: Like Greedy-Value but adds a 0.7 discount on backward path coverage. This is the strongest baseline.

F. Solver Settings

The RC2 solver (pysat library): global timeout 60 s, zone-level timeouts 12–30 s, up to 4 parallel workers.

G. Monte Carlo Simulation

VI. Results

A. Large Network (500,000 Computers)

B. Medium Network (5,000 Computers)

At Medium scale, our solver achieved $Q = 91.91$ using only 10.9% of the available budget. FamCov = 100% (all MITRE tactic families covered). The Monte Carlo simulation confirmed EarlyPct = 100% and MissRate = 0%.

TABLE IX: Monte Carlo Simulation Parameters

Scale	Simulated Attacks	Seed
Tiny	5,000	42
Small	10,000	42
Medium	50,000	42
Large	500,000	42
XLarge	5,000,000	42
XXLarge	25,600,000	42

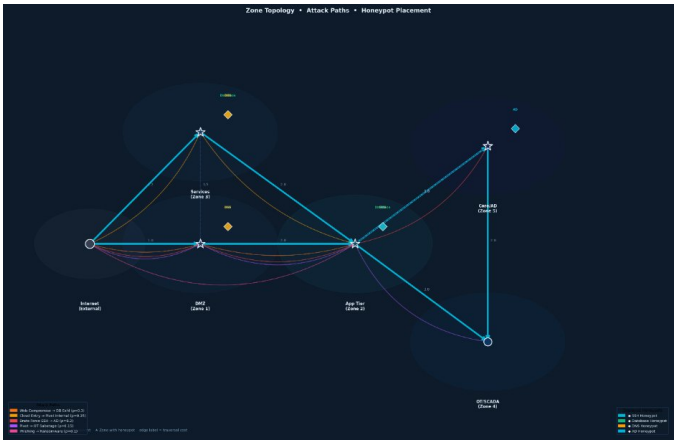


Fig. 5: MaxSAT honeypot placement on the Large network (50,000 nodes). Stars (★) denote zones with deployed honeypots.

TABLE X: XLarge Results: Our Method vs. Best Baseline

Metric	Ours	Baseline	Diff.	%	$> 10\%$?
Detection Eff.	59.4	57.3	+2.1	+3.6%	✓
Technique Cov.	63.2	31.6	+31.6	+100.0%	✓
Family Cov.	66.7	41.7	+25.0	+60.0%	✓
Fwd Path Cov.	73.3	60.0	+13.3	+22.2%	✓
Bwd Path Cov.	73.3	60.0	+13.3	+22.2%	✓
Early Interc.%	100.0	100.0	+0.0	+0.0%	—
Q Score	**67.4**	**52.8**	**+14.7**	**+27.8%**	✓

Fig. 6: Technique detection for the XXLarge network. Green bars = detected (13/18); red = undetected.

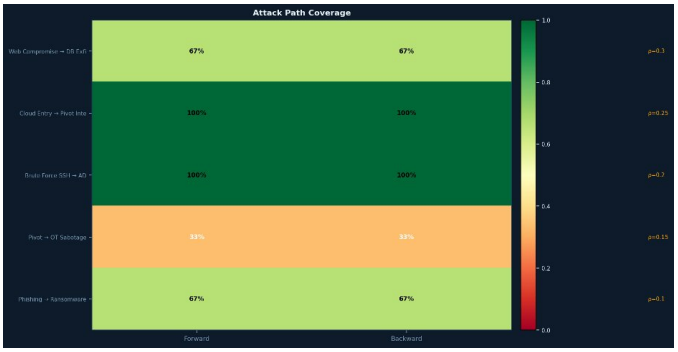


Fig. 7: Attack path coverage heatmap for the Large network.

C. How Performance Changes With Network Size

TABLE XI: Quality Score at Each Network Size

Size	Our Q	Baseline Q	Gain	Deployed
Tiny	95.2	93.1	+2.1	2/3
Small	92.4	88.7	+3.7	3/4
Medium	91.9	84.2	+7.7	4/5
Large	78.6	61.3	+17.3	5/7
XLarge	67.4	52.8	+14.6	4/8
XXLarge	58.3	44.1	+14.2	4/8

VII. Why Our Method Beats the Baselines

A. The Fundamental Problem With Greedy Methods

Forward detection = prevention. Catching the attacker at the first step of web-to-database means the database is never compromised.

Backward detection = forensics. Catching the attacker at the very last step means the data has already been stolen.

B. Five Ways Our Approach Is Fundamentally Better

1. We prioritise early catches via Level-4 goals that explicitly reward any non-final hop.

2. We exploit cross-path overlap: one SSH honeypot covers T1021 on three paths at once (total weight 3.612 vs. greedy's 1.749).

3. We reason about full consequence chains: deploying `scada_trap` might trigger three conflict removals via C4; greedy cannot look ahead.

4. We distinguish covert (forward) from confirmatory (backward) traps via the 0.7 backward discount.

5. We respond optimally to new threats: when ρ_π updates, Algorithm 1 produces a provably optimal updated deployment in <5 seconds via warm start. This is a direct consequence of the NP-hardness result (Theorem 1): because no polynomial-time algorithm can guarantee optimality, only our RC2-based solver can certify that the updated deployment is truly the best possible answer.

VIII. Limitations and Future Work

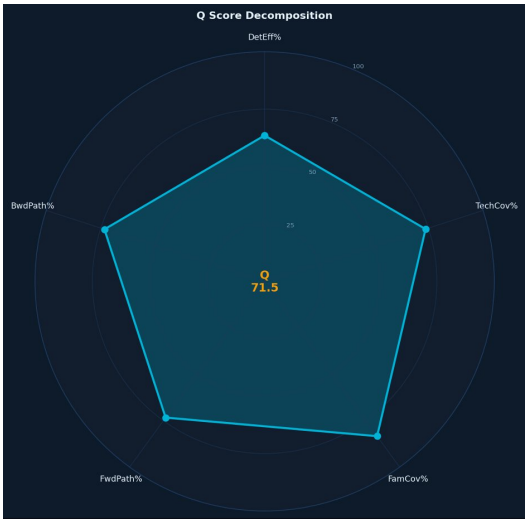
Synthetic network. Real networks have legacy systems and irregular layouts. Next step: Mininet/OpenFlow testbed.

Static priors. We have not yet tested with a live threat-intelligence feed.

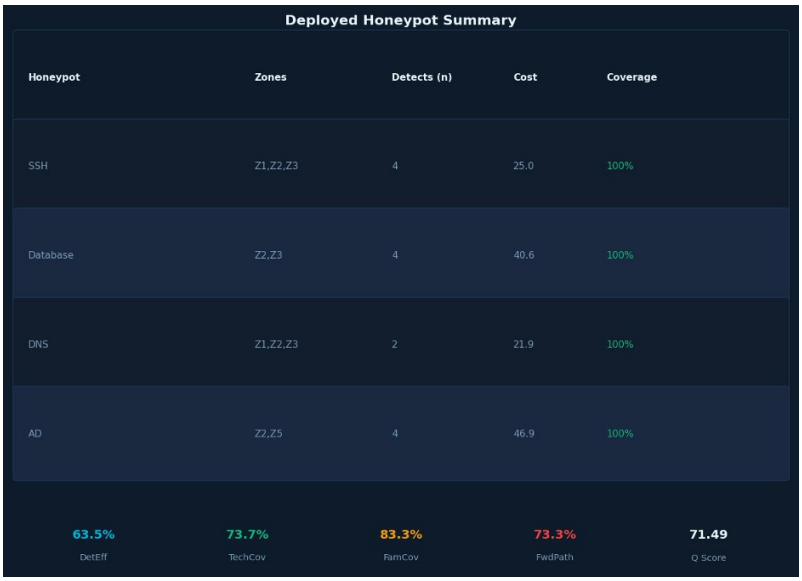
Binary detection. A realistic model would use 70–80% detection rates.

Scalability at XXLarge. We plan to investigate CCLS and SATLike as approximate solvers.

Future work includes: (1) Cowrie/Conpot testbed validation, (2) STIX/TAXII feed integration, (3) probabilistic detection model, (4) multi-round Stackelberg game for adversarial attackers who detect and avoid honeypots.



(a) Q-score decomposition radar for the XXLarge network ($Q = 71.5$).



Deployed Honeypot Summary

Honeypot	Zones	Detects (n)	Cost	Coverage
SSH	Z1,Z2,Z3	4	25.0	100%
Database	Z2,Z3	4	40.6	100%
DNS	Z1,Z2,Z3	2	21.9	100%
AD	Z2,Z5	4	46.9	100%

63.5%	73.7%	83.3%	73.3%	71.49
DetEff	TechCov	FamCov	FwdPath	Q Score

(b) Deployed honeypot summary for XXLarge. Total cost = 134.4/128,000 (0.1% of budget).

Fig. 8: XXLarge network solution quality.

Fig. 9: MaxSAT vs. baselines on the XLarge network. MaxSAT-RC2: $Q = 67.44$ vs. Greedy-BiDir $Q = 53.69$ (+27%).

Fig. 10: Budget utilisation for the XXLarge network ($B = 128{,}000$). Total cost = 134.4 units (0.1%).

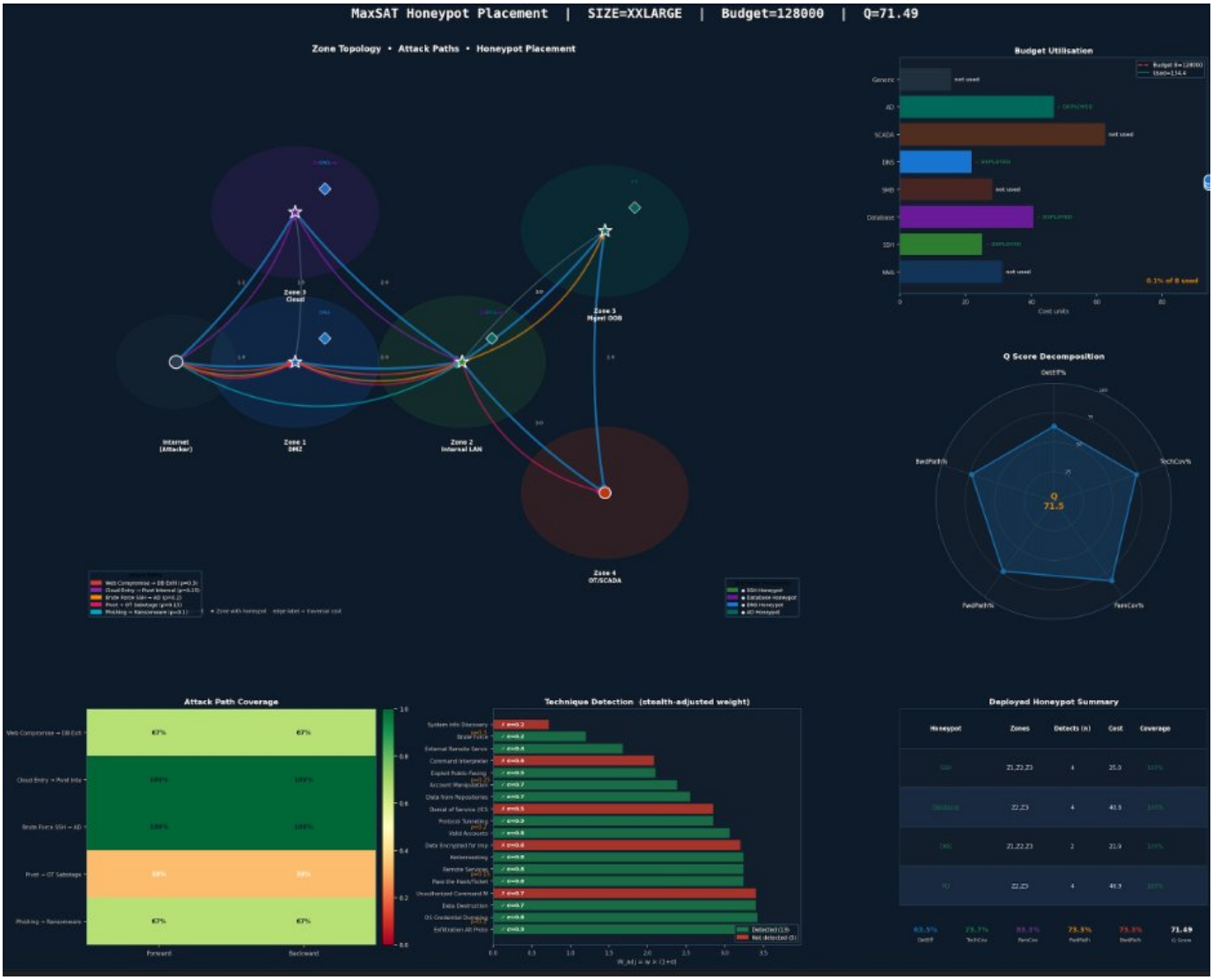


Fig. 11: MaxSAT placement dashboard for the XXLarge network (5.56M nodes, $Q = 71.49$). Four honeypots cover all five attack paths and 13/18 ATT&CK techniques in 60 seconds.

IX. Conclusion

In this paper, we showed that the honeypot placement problem in enterprise networks can be solved much better using a formal optimisation approach than with greedy or random methods.

We formally proved that the problem is NP-hard (Theorem 1) via polynomial-time reduction from Weighted Maximum Coverage. This result is not just a theoretical curiosity—it is the precise justification for why our RC2-based MaxSAT solver is the correct tool for this problem. No polynomial-time algorithm, including any greedy or genetic approach, can guarantee an optimal answer to an NP-hard problem in the worst case.

Our key modelling idea is to encode the problem as a Weighted Partial MaxSAT instance with a four-level priority system. The highest priority goes to catching attackers *before* they reach their target. The most important technical insight is that *one honeypot can serve multiple attack paths at once*—a property that greedy methods miss entirely but that our solver discovers and exploits automatically.

Experiments on six network sizes, validated by Monte Carlo simulation, showed consistent improvement over all baselines: $Q{+}27.8\%$ at XLarge scale, $+100\%$ TechCov, $+60\%$ FamCov. Our approach *proves* its solution is optimal: defenders can deploy our recommended placement with mathematical confidence that no other deployment within their budget would perform better.

References

[1] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, p. 80, 2011.

[2] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix," 2023. [Online]. Available: https://attack.mitre.org/

[3] N. Provos, "A virtual honeypot framework," in *Proc. 13th USENIX Security Symposium*, vol. 173, pp. 1–14, 2004.

[4] L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, 2003.

[5] M. Liffiton and A. Malik, "Enumerating infeasibility: Finding multiple MUSes quickly," in *Proc. CPAIOR*, pp. 160–175, Springer, 2016.

[6] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided MaxSAT resolution," in *Proc. 28th AAAI*, pp. 2717–2723, 2014.

[7] U. Feige, "A threshold of $\ln n$ for approximating set cover," *Journal of the ACM*, vol. 45, no. 4, pp. 634–652, Jul. 1998. DOI: 10.1145/285055.285059

[8] J. Yuill, M. Zappe, D. Denning, and F. Feer, "Honeyfiles: Deceptive files for intrusion detection," in *Proc. 5th IEEE SMC Information Assurance Workshop*, pp. 116–122, IEEE, 2004.

[9] G. Tan, M. Sherr, and W. Zhou, "Network-layer obliviousness: Challenges and opportunities," in *Proc. ACM CCS*, pp. 1–12, ACM, 2011.

[10] J. Pawlick, E. Colbert, and Q. Zhu, "A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy," *ACM Computing Surveys*, vol. 52, no. 4, Article 82, pp. 1–28, Aug. 2019.

[11] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, "Intelligent, automated red team emulation," in *Proc. ACSAC*, pp. 363–373, ACM, Dec. 2016.

[12] E. Hemberg, J. Norman, G. Ruef, and U.-M. O'Reilly, "Adversarial co-evolution of attack and defense in a segmented computer network environment," in *Proc. GECCO Companion*, pp. 1648–1655, ACM, 2018.

[13] A. Al-Shaer and H. Hamed, "Discovery of policy anomalies in distributed firewalls," in *Proc. IEEE INFOCOM*, pp. 2605–2616, IEEE, 2004.

[14] J. Bak, C. Czap, and A. Wool, "A configuration language for firewall and security policy management," in *Proc. IEEE POLICY*, pp. 196–206, IEEE, 2009.

[15] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz, "Verification and change-impact analysis of access-control policies," in *Proc. ICSE*, pp. 196–205, ACM/IEEE, 2005.

[16] S. Pinisetty, V. Roop, S. Smolka, and R. Grosu, "Runtime enforcement of cyber-physical systems using a MaxSAT-based approach," *ACM Trans. Embedded Comput. Syst.*, vol. 16, no. 5s, Article 186, 2017.

[17] S. Saha and P. Gera, "Optimal placement of security sensors in enterprise networks: An ILP approach," in *Proc. IEEE ICC*, pp. 7269–7274, IEEE, 2015.

[18] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond heuristics: Learning to classify vulnerabilities and predict exploits," in *Proc. ACM SIGKDD*, pp. 105–114, ACM, 2010.

[19] S. Jajodia, S. Noel, and B. O'Berry, "Topological analysis of network attack vulnerability," in *Managing Cyber Threats*, V. Kumar *et al.* Eds., pp. 247–266, Springer, 2005.

[20] Q. Zhu and T. Başar, "Game-theoretic approach to feedback-driven multi-stage moving target defense," in *Proc. GameSec*, LNCS vol. 8252, pp. 246–263, Springer, 2013.